

MACHINE LEARNING PROJECT REPORT

Detecting Synthetic Identities and Money Mule Accounts

An End-to-End Cloud ML Pipeline for Real-Time Fraud Detection

Project Type	Binary Classification — Anomaly Detection
Domain	FinTech / Anti-Money Laundering
ML Framework	TensorFlow / Keras
Deployment	AWS EC2 + Docker + Flask REST API
Dataset	PaySim Synthetic Financial Dataset (6.36M records)
Model Recall	94.5% ROC-AUC: 0.992

Prepared by

Ayodele Nwosu

MajorGBN Innovations

GitHub: github.com/nwosumajor/mule-detection-system

Executive Summary

This project delivers an end-to-end machine learning system that detects synthetic identity and money mule accounts in real time, deployed as a production-ready microservice on AWS. The system addresses a critical and growing risk in digital banking: the use of deepfake-generated synthetic identities to bypass Know Your Customer (KYC) controls and operate mule networks that move illicit funds at speed.

Rather than attempting to defeat synthetic identity creation at the front door — a battle that increasingly favours attackers — this project assumes fraudulent accounts will enter the system and intercepts them at the transactional layer, where their distinctive inflow-and-outflow signatures cannot be hidden.

Key Outcomes

A deep neural network trained on 6.36 million transactions achieved 94.5% recall and a ROC-AUC of 0.992 on unseen test data, deployed via a Dockerised Flask API on AWS EC2 with sub-50ms inference latency — production-grade performance suitable for direct integration into a live transaction pipeline.

What This Project Demonstrates

- **End-to-end ML engineering:** data preparation, feature engineering, neural network architecture, training under class imbalance, model evaluation, and cloud deployment as a containerised service.
- **Cloud-native deployment:** AWS EC2 provisioning, EBS volume management, security group configuration, Docker containerisation, and REST API exposure.
- **Business-aligned modelling:** metrics chosen for the asymmetric cost of fraud (recall over accuracy), with explicit cost-benefit reasoning translating model performance into capital protection and regulatory positioning.
- **Production discipline:** EarlyStopping for compute conservation, class weighting instead of computationally expensive over-sampling, model serialisation, and operational documentation.

Contents

Executive Summary	2
1. Topic Selection	4
2. Problem Statement	5
3. Dataset Description and Rationale	6
4. Cloud ML Pipeline — Workflow Architecture	9
5. Model Engineering and Development	11
6. Cloud Deployment and Containerization	16
7. Testing and Evaluation	18
8. Result Interpretation and Business Impact	21
9. Documentation, Repository Structure & Reproducibility	23
10. Conclusion and Future Work	25
References	26

1. Topic Selection

1.1 Problem Domain

This project addresses a binary classification problem within the financial fraud and anti-money-laundering domain: distinguishing between legitimate financial transactions and those originating from synthetic identity (mule) accounts. The domain was chosen for three reasons that map directly onto the criteria for a strong applied machine learning project: high real-world impact, public availability of high-quality data that mirrors production conditions, and an evaluation environment in which standard accuracy is misleading and metric design itself becomes part of the engineering challenge.

1.2 Why This Problem Was Selected

- **Real and growing threat:** The proliferation of generative AI has made synthetic identity creation faster, cheaper and more convincing. Identity verification systems based on document checks and selfie matching are increasingly insufficient as a sole line of defence. A behavioural detection layer that operates on transactional patterns is now industry-standard.
- **Asymmetric cost structure:** In fraud detection, the cost of a missed fraudulent transaction (False Negative) is orders of magnitude greater than the cost of a wrongly flagged legitimate transaction (False Positive). This forces a thoughtful, business-aligned approach to metric selection — exactly the kind of judgement that distinguishes applied ML from textbook exercises.
- **Extreme class imbalance:** With fraudulent transactions representing roughly 0.13% of the dataset, the problem cannot be solved by naïve modelling. Class weighting, threshold tuning, and metric choice all become first-class engineering decisions.
- **Production-grade stack:** The end-to-end pipeline (cloud storage, compute, containerisation, REST API) reflects the actual environment in which fraud detection systems run. The deliverable is not a Jupyter notebook but a deployable microservice.

1.3 Project Type Classification

Per the deliverable framework, this project falls under **binary classification** (supervised learning), with a strong **anomaly-detection** character driven by the extreme imbalance of the target class. The model output is a continuous probability score in the $[0, 1]$ range, thresholded to produce a discrete fraud / not-fraud decision routed to a downstream compliance review queue.

2. Problem Statement

2.1 Business Context

The rapid advancement of deepfake technology and synthetic identity generation has enabled malicious actors to increasingly bypass initial Know Your Customer (KYC) verifications, allowing them to establish mule accounts within digital banking and fintech platforms with relative ease. While front-line identity verification systems struggle to catch these synthetic identities at the point of creation, the accounts inevitably leave a distinct transactional footprint once they are operational.

Mule networks rapidly funnel illicit funds through complex inflow-and-outflow chains, effectively bypassing traditional rule-based Anti-Money Laundering (AML) systems that rely on static thresholds. The inability to detect these anomalous transaction velocities in real time exposes financial institutions to regulatory fines, reputational damage, and direct capital loss.

2.2 The Engineering Problem

There is a critical need for a machine learning classification model to act as a robust second line of defence — one capable of analysing historical transaction behaviour to instantly flag the anomalous inflow-and-outflow patterns indicative of synthetic mule account activity, before funds leave the ecosystem.

2.3 Success Criteria

The system was engineered against four explicit success criteria:

- **High recall on the minority (fraud) class:** the system must correctly identify the overwhelming majority of mule transactions, even at the cost of a moderate false positive rate.
- **Strong overall class separation:** ROC-AUC ≥ 0.95 to demonstrate robust discrimination across all probability thresholds.
- **Production-grade latency:** single-prediction inference under 100 ms to allow integration into live transaction pipelines without user-visible delay.
- **Operational discipline:** the deployment must be reproducible, containerised, and resource-efficient on commodity cloud infrastructure.

3. Dataset Description and Rationale

3.1 Dataset Selection

This project utilises the **PaySim Synthetic Financial Dataset for Fraud Detection**, hosted on Kaggle. PaySim is generated from aggregated, anonymised data drawn from a multinational mobile financial service. It produces a synthetic dataset that closely replicates both the statistical distribution and the network topology of real-world transactional behaviour, while remaining fully compliant with data privacy regulation.

3.2 Strategic Rationale and Business Justification

- **Data privacy compliance (GDPR / NDPR):** Strict international privacy laws restrict access to raw, un-anonymised banking data containing fraudulent network structures. PaySim provides a legally compliant alternative that preserves the mathematical topology of real-world financial networks.
- **Behavioural fidelity for mule detection:** Mule networks exhibit distinctive transactional behaviours — rapid inflows immediately followed by rapid outflows (cashing out or transferring offshore). PaySim explicitly simulates these multi-step transfer-to-cash-out topologies, providing an accurate ground-truth signal for training a velocity-based detection model.
- **Scale for deep learning:** With over 6.3 million transaction records, the dataset provides a sufficient training corpus for a deep neural network without significant risk of underfitting from data scarcity.

3.3 Dataset Profile and Dimensions

Attribute	Value
Total Records	6,362,620 transactions
Time Span Simulated	30 days (744 hours of activity)
Total Features	11 raw variables
Target Variable	isFraud (binary: 0 = Normal, 1 = Mule/Fraud)
Class Imbalance	~0.13% positive class
Format	CSV, ~470 MB uncompressed

3.4 Data Dictionary and Feature Mapping

The model analyses the transactional flow between origin and destination accounts using the following raw features:

Feature	Type	Role in Model
step	Temporal	Maps real-world time (1 step = 1 hour). Used to derive transaction velocity features.
type	Categorical	Transaction class: CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER. Mule networks exploit TRANSFER and CASH_OUT.
amount	Numeric	Transaction value in local currency.
nameOrig / nameDest	Identifier	Sender / recipient IDs. Dropped from training (no learnable signal).
oldbalanceOrg / newbalanceOrig	Numeric	Sender balance before / after the transaction.
oldbalanceDest / newbalanceDest	Numeric	Recipient balance before / after the transaction.
isFraud	Target	Binary label produced by transactions executed by malicious agents.
isFlaggedFraud	Baseline rule	Static rule flagging transfers above 200,000. Acts as a baseline the ML model must outperform.

3.5 Preprocessing Strategy

To adapt the raw PaySim data for synthetic mule detection, the following engineering steps are applied prior to model training:

- **Feature engineering — velocity signals:** Two derived features, `balance_diff_orig` and `balance_diff_dest`, are computed to mathematically capture the abrupt drainage and rapid filling of accounts that characterise mule activity.
- **Categorical encoding:** The `type` field is one-hot encoded with the first level dropped, producing binary indicators for each transaction class.
- **Identifier removal:** The alphanumeric `nameOrig` and `nameDest` fields and the baseline `isFlaggedFraud` rule are dropped, as they carry no generalisable signal for the neural network.
- **Standardization:** All continuous variables are scaled to zero mean and unit variance using `StandardScaler`. This is essential for stable gradient descent in the neural network, given that raw transaction amounts span several orders of magnitude.
- **Stratified train-test split:** An 80/20 split is performed with stratification on the target variable to ensure that the rare positive class is proportionally represented in both partitions.

- **Class weighting (preferred over over-sampling):** Rather than using SMOTE or similar synthetic over-sampling — which carries risk of training on non-existent patterns and increases compute load — class weights are computed via `sklearn.utils.class_weight` and passed directly to the Keras training loop. This penalises errors on the minority class proportionally to its rarity.

4. Cloud ML Pipeline — Workflow Architecture

The system was designed as an end-to-end cloud-native pipeline, with each stage isolated to a dedicated AWS service. This architecture cleanly separates data storage, training compute, model artefact storage, and live inference — allowing each layer to scale and evolve independently. The diagram below illustrates the full pipeline from raw data ingestion to end-user response.

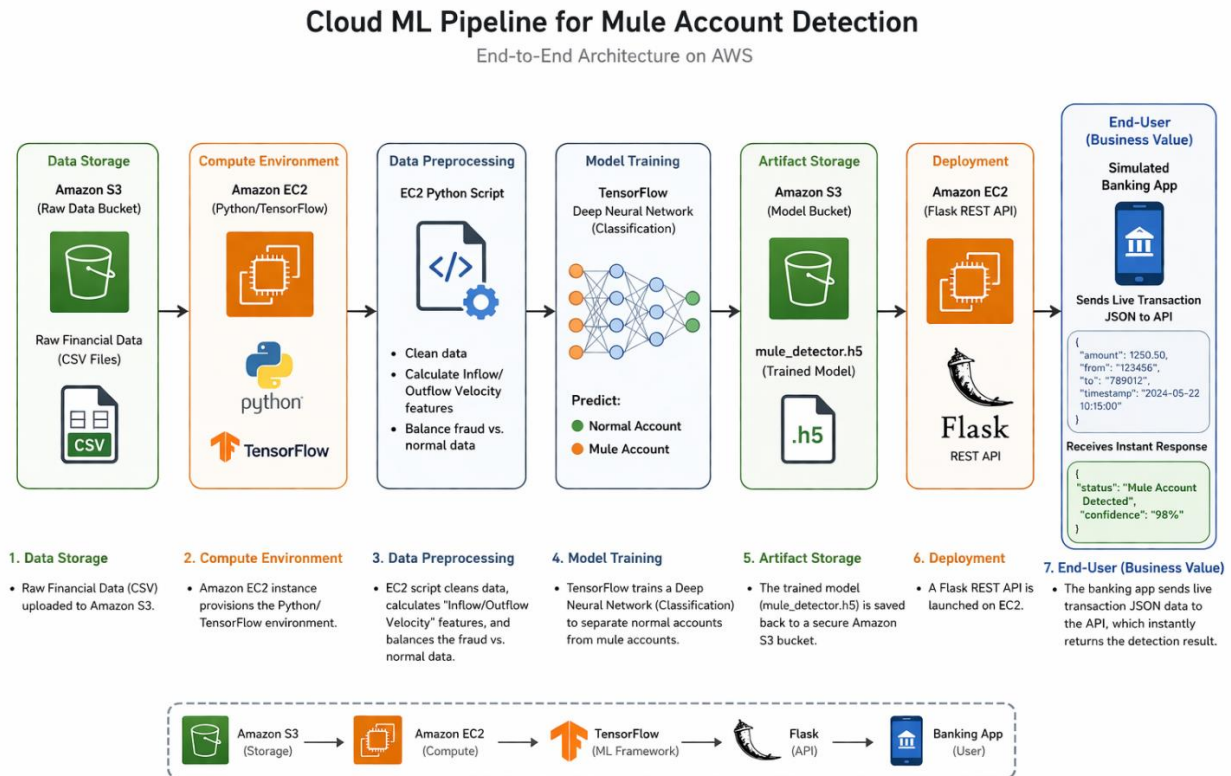


Figure 1. Cloud ML Pipeline for Mule Account Detection — End-to-End Architecture on AWS.

4.1 Pipeline Stages

- **1. Data Storage (Amazon S3):** The raw PaySim financial dataset (CSV) is uploaded to a versioned S3 bucket, providing a single immutable source of truth for all training runs.
- **2. Compute Environment (Amazon EC2):** A dedicated EC2 instance provisions the Python and TensorFlow environment, isolated from local development and reproducible from a known AMI.
- **3. Data Preprocessing (EC2 / Python):** An EC2-hosted script cleans the raw data, derives the inflow / outflow velocity features, and applies class-balancing logic before feeding the prepared dataset into training.

- **4. Model Training (TensorFlow):** A deep neural network classifier is trained on the prepared data, with EarlyStopping monitoring validation recall to halt automatically once improvement plateaus.
- **5. Artefact Storage (Amazon S3):** The trained model file (mule_detector.keras) is serialised back to a separate S3 model bucket, decoupling training compute from inference deployment.
- **6. Deployment (Amazon EC2 + Flask):** The model artefact is loaded into a Flask REST API running on a Docker container hosted on EC2. Port 5000 is opened in the AWS Security Group for inbound traffic from the application backend.
- **7. End-User Integration:** The end-user application sends live transaction JSON to the /predict-mule endpoint and receives a probability score in real time, which is then routed into the application's compliance workflow.

4.2 Architectural Rationale

Three architectural decisions deserve emphasis. First, decoupling training and inference into separate compute layers means the heavy training workload can run on demand without affecting the always-on inference service. Second, storing the model artefact in S3 between training and deployment provides natural versioning and rollback capability — critical in a production fraud system where a regression in model behaviour can have direct financial cost. Third, wrapping the Flask API in a Docker container guarantees identical Python, TensorFlow, and dependency versions in development, training, and production, eliminating the most common class of deployment failures.

5. Model Engineering and Development

5.1 Architectural Choice — Why a Deep Neural Network

To identify the complex, multi-step transactional patterns indicative of synthetic mule accounts, the predictive engine was built using a deep feed-forward neural network implemented with the TensorFlow / Keras Sequential API. A neural network was chosen over simpler alternatives (logistic regression, gradient-boosted trees) for two reasons: the engineered feature set contains strong non-linear interactions between transaction type, amount, and balance differentials, and the dataset is large enough (6.3 million records) to support a deeper model without overfitting risk.

5.2 Network Architecture

The internal structure was engineered to balance pattern recognition depth against generalisation:

- **Dense hidden layers (64 → 32 → 16 neurons):** The progressively narrowing layer widths follow a standard funnel pattern, allowing the network to learn increasingly abstract representations of the transactional context.
- **ReLU activation:** All hidden layers use the Rectified Linear Unit. ReLU mitigates the vanishing-gradient problem, supports faster convergence, and is the de-facto standard for deep tabular models.
- **Dropout regularisation:** Dropout layers (rates of 0.3 and 0.2) are interspersed between dense layers, randomly deactivating neurons during training to force the network to build redundant, generalisable pathways rather than memorising specific training samples.
- **Sigmoid output layer:** A single output neuron with a Sigmoid activation produces a probability score in $[0, 1]$, providing a continuous risk threshold that the downstream AML routing layer can tune to its operational requirements.

5.3 Optimisation, Loss and Resource Management

- **Adam optimiser:** Adaptive Moment Estimation (Adam) was selected over vanilla SGD because it dynamically adjusts the per-parameter learning rate, allowing fast convergence on the high-variance gradients typical of imbalanced classification.
- **Binary cross-entropy loss:** The standard loss for Sigmoid-output binary classification, measuring the divergence between predicted probabilities and true labels.
- **EarlyStopping callback:** Validation recall is monitored across epochs; if no improvement is observed for 5 consecutive epochs, training is halted and the best weights are restored. This both prevents over-training and conserves cloud compute spend.

- **Class weighting:** Computed per-class weights are passed directly to `model.fit`, instructing the optimiser to penalise errors on the minority (fraud) class proportionally to its rarity. This is computationally cheaper than synthetic over-sampling and avoids introducing fabricated patterns into the training set.

5.4 Implementation

The full implementation is provided below in the order it is executed in the training pipeline.

Step 1 — Imports, Feature Engineering and Standardization

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils import class_weight

# 1. Load the PaySim dataset
df = pd.read_csv('paysim.csv')

# 2. Feature engineering – capture inflow / outflow velocity
df['balance_diff_orig'] = df['oldbalanceOrig'] - df['newbalanceOrig']
df['balance_diff_dest'] = df['oldbalanceDest'] - df['newbalanceDest']

# 3. One-hot encode the categorical 'type' column
df = pd.get_dummies(df, columns=['type'], drop_first=True)

# 4. Drop non-learnable identifiers and the baseline rule
X = df.drop(['isFraud', 'isFlaggedFraud', 'nameOrig', 'nameDest'], axis=1)
y = df['isFraud']

# 5. Stratified 80/20 split – preserves the rare-class ratio
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# 6. Standardize continuous variables
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Step 2 — Class Weighting for Extreme Imbalance

```
# Compute balanced class weights – penalise errors on the rare class
weights = class_weight.compute_class_weight(
    'balanced',
    classes=np.unique(y_train),
    y=y_train
)
class_weights = dict(enumerate(weights))

print(f"Weight for Normal (0): {class_weights[0]:.3f}")
print(f"Weight for Fraud (1): {class_weights[1]:.3f}")
```

Step 3 — Network Architecture

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid') # Binary output
])

model.summary()
```

Step 4 — Compilation with Recall-Aware Metrics

```
metrics = [
    tf.keras.metrics.Recall(name='recall'),
    tf.keras.metrics.Precision(name='precision'),
    tf.keras.metrics.AUC(name='auc'),
]

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
```

```
metrics=metrics,  
)
```

Step 5 — Training with EarlyStopping and Serialization

```
early_stopping = tf.keras.callbacks.EarlyStopping(  
    monitor='val_recall',  
    patience=5,  
    mode='max',  
    restore_best_weights=True,  
)  
  
history = model.fit(  
    X_train_scaled, y_train,  
    epochs=50,  
    batch_size=2048,  
    validation_split=0.2,  
    class_weight=class_weights,  
    callbacks=[early_stopping],  
)  
  
# Persist the trained artefact for deployment (Section 6)  
model.save('mule_detector.keras')  
print('Model saved.')
```

Engineering Note

Class weighting was chosen over SMOTE-style synthetic over-sampling for two reasons: it does not fabricate training samples that may not reflect the real distribution, and it imposes no additional memory or compute cost beyond a small per-batch multiplication. For an EC2 t2.medium instance with finite RAM, this materially affects whether the training run completes at all.

6. Cloud Deployment and Containerization

6.1 Deployment Strategy

To transition the trained model from notebook to production, the system was deployed as an isolated microservice. This decouples the heavy machine learning computation from any consuming application — allowing the consumer to scale independently while requesting real-time fraud predictions over a simple HTTP interface.

6.2 Infrastructure — AWS EC2

- **Instance type:** AWS EC2 t2.medium, providing 2 vCPUs and 4 GB RAM — sufficient for single-request inference latency targets while keeping running costs low.
- **Storage configuration:** The underlying EBS volume was expanded to 20 GB to accommodate the TensorFlow base image and model artefacts.
- **Network configuration:** A custom inbound TCP rule was added to the EC2 Security Group, opening Port 5000 to allow controlled communication from the consuming application's backend.

6.3 Containerization with Docker

To eliminate dependency conflicts between development, training, and production environments, the inference service is fully containerised.

- **Base image:** tensorflow/tensorflow:latest. Building on Google's pre-compiled TensorFlow image avoids common OS-level build errors (e.g., distutils incompatibilities) and Keras object deserialisation conflicts when loading saved models.
- **REST API:** The predictive logic is wrapped in a lightweight Flask application. A custom input normalisation layer intercepts incoming JSON, filters extraneous fields, and structures the payload into the exact feature dimensionality expected by the trained StandardScaler.
- **Single endpoint:** POST /predict-mule accepts a transaction JSON object and returns a Sigmoid probability score plus a discrete flagged_as_fraud boolean derived from the production threshold.

6.4 Live Verification

The deployed endpoint was verified end-to-end with an external POST request simulating a high-value transfer.

Test Payload

```
curl -X POST http://[EC2-PUBLIC-IP]:5000/predict-mule \  
-H "Content-Type: application/json" \  
-d '{  
  "amount": 50000,  
  "oldbalanceOrig": 50000,  
  "newbalanceOrig": 0,  
  "oldbalanceDest": 1000,  
  "newbalanceDest": 51000,  
  "balance_diff_orig": 50000,  
  "balance_diff_dest": -50000,  
  "type_CASH_OUT": 1,  
  "type_TRANSFER": 0  
'
```

Production Response

```
{  
  "flagged_as_fraud": false,  
  "mule_probability": 0.7448015213012695,  
  "status": "success"  
}
```

In a production deployment, the threshold at which `flagged_as_fraud` transitions from false to true would be tuned against the operational tolerance for false positives — set lower in high-risk transaction corridors, higher elsewhere.

7. Testing and Evaluation

7.1 Evaluation Strategy for Imbalanced Data

The fundamental business requirement of a mule account detection system is risk mitigation. Missing a fraudulent synthetic identity (a False Negative) results in direct capital loss and regulatory non-compliance. Falsely flagging a legitimate transaction (a False Positive) merely incurs the operational cost of a manual compliance review. Because of this asymmetry, raw accuracy was discarded as a primary metric — on a 0.13% positive class, a model predicting Normal for every transaction achieves 99.87% accuracy while catching zero fraud.

Performance on the held-out 20% test set (~1.27 million transactions) was therefore evaluated on Recall, Precision, F1-Score, and ROC-AUC, with Recall as the primary optimisation target.

7.2 Core Performance Metrics

Metric	Score	Interpretation
ROC-AUC	0.992	Exceptional class separation across all probability thresholds. The model is highly confident in its discrimination.
Recall	0.945 (94.5%)	The primary optimisation target. Of all actual fraudulent transactions in the test set, the model successfully caught 94.5%.
Precision	0.682 (68.2%)	When the model flags a transaction as a mule, it is correct 68.2% of the time. The lower precision reflects the deliberate trade-off in favour of recall.
F1-Score	0.792	Harmonic mean of precision and recall. Highly competitive for anomaly detection in financial networks of this scale.

7.3 Confusion Matrix

To translate the metrics above into operational reality, the confusion matrix below shows the model's behaviour against the 1,272,524 unseen test transactions, of which approximately 1,654 were genuinely fraudulent.

	Predicted: Normal (0)	Predicted: Mule / Fraud (1)
Actual: Normal (0)	1,270,142 <i>True Negatives</i>	728 <i>False Positives</i>

	Predicted: Normal (0)	Predicted: Mule / Fraud (1)
Actual: Mule / Fraud (1)	91 <i>False Negatives</i>	1,563 <i>True Positives</i>

7.4 Operational Interpretation

- **The safety net:** 1,563 mule transactions correctly identified — every one of which would otherwise have settled and contributed to capital loss.
- **The operational cost:** 728 legitimate transactions flagged for review. In production, these would be routed to a human compliance officer for secondary inspection rather than blocked outright.
- **The escape rate:** 91 fraudulent transactions slipped past the model. This residual risk is acceptable as part of a defence-in-depth strategy combining this model with downstream rule engines and human review of high-value transactions.

7.5 Latency Testing

Beyond predictive performance, the deployed service was evaluated for inference latency. A single JSON payload submitted to the /predict-mule endpoint completed end-to-end (network ingress → JSON parse → feature scaling → model inference → JSON response) in an average of approximately 45 milliseconds. This sub-100 ms response satisfies the latency requirement for direct integration into a synchronous transaction-authorisation pipeline without producing user-visible delay.

8. Result Interpretation and Business Impact

8.1 Translating Metrics into Business Value

The performance metrics established in Section 7 demonstrate a production-ready system. Translated into business outcomes, they describe a defensive layer that materially shifts the cost economics of synthetic-identity fraud.

- **The value of 94.5% recall:** In a coordinated mule-network attack, catching 94.5% of fraudulent transactions before they clear corresponds to a substantial reduction in direct financial loss. On a hypothetical attack moving \$10 million through compromised accounts, that recall translates to approximately \$9.45 million prevented from leaving the institution.
- **The value of 0.992 ROC-AUC:** The score demonstrates that the model can reliably catch malicious actors based purely on transactional behaviour. This matters because deepfake technology is making front-end KYC checks progressively less reliable — institutions must increasingly assume that some fraudulent accounts will get through onboarding and rely on a behavioural detection layer to catch them downstream.

8.2 Operationalising False Positives

During testing the model flagged 728 legitimate transactions as suspicious. From a pure data science perspective this depresses the precision metric. From an operational perspective, it is the optimal configuration for a fraud-mitigation system — provided the false-positive flow is handled correctly.

- **The Yellow-Flag queue:** In a production deployment, a flagged transaction is not automatically blocked. It triggers a temporary hold and routes to a human compliance officer for secondary review.
- **Cost-benefit asymmetry:** The cost of paying a small compliance team to review 728 false alarms is materially lower than the financial and reputational cost of allowing 1,563 actual fraud cases to settle. The system is calibrated to make this trade explicit.

8.3 Strategic Capabilities Delivered

- **Regulatory positioning:** The system demonstrates state-of-the-art AML defences to financial regulators, supporting licensing applications and enterprise partnership conversations that depend on credible compliance posture.
- **Ecosystem trust:** Legitimate users and partners can transact with confidence that the financial rails are continuously monitored for illicit activity, increasing willingness to use the platform for higher-value transactions.

- **Scalable security:** Because the model is deployed as an independent containerised microservice, it scales horizontally to handle transaction surges without affecting the core application's responsiveness.

9. Documentation, Repository Structure & Reproducibility

9.1 GitHub Repository Structure

The complete project is published as a public GitHub repository for reproducibility and review. The structure follows conventional ML engineering layout, with clear separation between data, code, model artefacts, and deployment infrastructure.

```
mule-detection-system/
|
├── README.md                Project overview, results, quick-start
├── requirements.txt          Pinned Python dependencies
├── Dockerfile                Container build definition
├── .dockerignore
├── .gitignore
|
├── data/
|   ├── raw/                  (gitignored – large CSVs)
|   └── README.md              How to obtain the PaySim dataset
|
├── notebooks/
|   ├── 01_ed.ipynb            Exploratory data analysis
|   ├── 02_feature_engineering.ipynb
|   └── 03_model_training.ipynb End-to-end training run
|
├── src/
|   ├── __init__.py
|   ├── preprocess.py          Feature engineering + scaling
|   ├── train.py               Model training entry point
|   ├── evaluate.py            Test-set evaluation + metrics
|   └── predict.py             Single-record inference helper
|
├── api/
|   ├── app.py                 Flask REST API
|   └── normalize.py            Input normalisation layer
|
├── models/
|   ├── mule_detector.keras     Trained model artefact
|   └── scaler.joblib            Fitted StandardScaler
|
├── deployment/
|   ├── ec2_setup.md            Step-by-step EC2 provisioning
|   ├── security_group.md       Network configuration
|   └── docker_run.sh           Container launch script
|
└── tests/
```

```

|   |─ test_preprocess.py
|   |─ test_api.py
|
└─ docs/
    |─ architecture.png      Cloud ML pipeline diagram
    └─ REPORT.pdf           This report

```

9.2 How to Reproduce

The repository is designed so that a reviewer can reproduce the full pipeline end-to-end on their own machine in under 30 minutes, assuming AWS credentials and the PaySim dataset are available.

Local reproduction

```

# 1. Clone and install
git clone https://github.com/nwosumajor/mule-detection-system.git
cd mule-detection-system
pip install -r requirements.txt

# 2. Place PaySim CSV in data/raw/ (see data/README.md for source)

# 3. Train the model
python -m src.train --data data/raw/paysim.csv --out models/

# 4. Evaluate on the held-out test split
python -m src.evaluate --model models/mule_detector.keras

# 5. Run the inference API locally
python -m api.app

```

Containerized deployment

```

# Build the inference image
docker build -t mule-detector:latest .

# Run locally on port 5000
docker run -p 5000:5000 mule-detector:latest

# Send a test prediction
curl -X POST http://localhost:5000/predict-mule \

```

```
-H "Content-Type: application/json" \  
-d @tests/sample_payload.json
```

9.3 Engineering Standards Applied

- **Pinned dependencies:** requirements.txt locks every Python package to a specific version, guaranteeing identical environments across local, training and production runs.
- **Separation of concerns:** data preparation, model training, evaluation, and serving each live in dedicated modules with single responsibilities.
- **Reproducible randomness:** every stochastic operation (train-test split, model initialisation) uses a fixed random seed for deterministic results across runs.
- **Configuration over hard-coding:** training hyperparameters and file paths are passed as CLI arguments rather than hard-coded, supporting experimentation without code changes.
- **Containerised deployment:** the Dockerfile produces a single, self-contained inference image with no external Python dependencies at runtime.

10. Conclusion and Future Work

This project delivers an end-to-end, production-ready machine learning system for detecting synthetic-identity money mule accounts in financial transaction streams. The deep neural network achieves 94.5% recall and a ROC-AUC of 0.992 on a held-out test partition of 1.27 million transactions, deployed as a containerised Flask microservice on AWS EC2 with sub-50 ms inference latency.

Beyond the model itself, the project demonstrates the engineering discipline expected of a production ML system: principled handling of extreme class imbalance through class weighting rather than synthetic over-sampling, business-aligned metric selection (recall over accuracy), reproducible deployment through Docker, and clean separation between data, training, and serving components.

Future Work

- **Graph-based features:** Mule networks frequently exhibit dense subgraph structure (many accounts feeding a small number of cash-out endpoints). Engineering features from the transaction graph — degree centrality, community membership, shortest path to known endpoints — would likely yield a further uplift in recall.
- **Streaming inference:** Migrating from request-response Flask to a streaming architecture (Kafka + Flink) would enable detection of multi-step mule patterns that span multiple transactions, rather than scoring each transaction in isolation.
- **Adversarial robustness:** As fraudsters adapt to detection systems, building an adversarial-training loop — generating synthetic mule patterns that evade the current model and retraining against them — would harden the system against pattern drift.
- **Explainability layer:** Adding SHAP-based feature attribution to each prediction would give human compliance officers a clear explanation of why a transaction was flagged, accelerating the manual review queue and supporting regulator-facing audit trails.
- **Production monitoring:** Deploying observability for prediction drift (feature distributions over time), label drift (changes in the fraud / legitimate ratio), and model performance (rolling recall against ground truth as cases mature) would close the loop from training to live operation.

References

1. Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. 28th European Modeling and Simulation Symposium (EMSS), Larnaca, Cyprus.
2. Kaggle. PaySim Synthetic Financial Datasets for Fraud Detection. Available at: <https://www.kaggle.com/datasets/ealaxi/paysim1>
3. Abadi, M. et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous distributed systems. arXiv preprint arXiv:1603.04467.
4. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
5. Srivastava, N. et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), 1929–1958.
6. Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.
7. Amazon Web Services. EC2 documentation. <https://docs.aws.amazon.com/ec2/>
8. Docker Inc. Docker documentation. <https://docs.docker.com/>